ASSIGNMENT 2.2 100Marks

PROBLEM STATEMENT

In **Distributed file sharing** a digital content is stored and accessed across multiple independent nodes or computers rather than a single central server. This architecture enhances availability and speed by allowing users to download pieces of a file from several sources simultaneously, reducing bottlenecks and points of failure.

Implement a single-threaded, decentralized file-sharing system in C. Each node in the system must maintain multiple TCP connections simultaneously and use a **Flooding Query** mechanism to locate files across the network. The system must remain entirely responsive to user input and network events without spawning multiple threads or processes.

<u>Example Scenario:</u>

Imagine you and 4 classmates are working on a project. Each of you has different documents, *code files*, and *presentations*. Instead of using a shared drive (which could fail), you build a P2P system where you can search for files across all your computers. If one computer goes offline, the important files are still available from others.

ASSIGNMENT 2.2 100 Marks

How to execute:

- For each peer run the peer code in a different terminals
- Create a folder containing files where each peers file are being stored and then when the peer searches for the file in the network and if it finds the file then it should receive the file from the file holder. After receiving, the file should be present in the peer's folder.
- Create a various different kinds topology p2p connections and analyse how the topology affects the amount of content replication for the topology.Show it clearly.

ASSIGNMENT 2.2 100Marks

Expected Outcome:

```
=== P2P Node Commands ===
1. connect <IP> <port>    - Connect to another peer
2. search <query>         - Search for files in network
3. get <filename>         - Download a file
4. list_peers             - Show connected peers
5. list_files             - Show shared files
6. share <filename>       - Share a file
7. unshare <filename>     - Stop sharing a file
8. reload                 - Reload shared files
9. quit                   - Exit program
Enter command: list_peers

=== Connected Peers (2) ===
==========================


=== P2P Node Commands ===
1. connect <IP> <port>    - Connect to another peer
2. search <query>         - Search for files in network
3. get <filename>         - Download a file
4. list_peers             - Show connected peers
5. list_files             - Show shared files
6. share <filename>       - Share a file
7. unshare <filename>     - Stop sharing a file
8. reload                 - Reload shared files
9. quit                   - Exit program
Enter command: share image1.jpg
File already in list, marked as shared: image1.jpg
```